

Final Report for STAT 451/551

Jack Barney, Kyle Maher, Max Donelan, & Tate Wietfeld

Spring 2026

Introduction

The given FP data contains 200 observations of eight predictors labeled $x1$ through $x8$, and the resulting response parameter y . Additional consideration was given to predicting y_bin which is 1 if y is greater than or equal to 70 and 0 otherwise. $x1$ through $x7$ are continuous while $x8$ is categorical with four groups A , B , C , and D . No context surrounding the predictors or response parameters was given. This report is segmented into initial Exploratory Data Analysis, followed by the Regression Models, then the Classification Models, and concludes with some overall observations. All code can be found in the following GitHub Repository <https://github.com/MaxDonelan/STAT-551-Final-Project>.

Exploratory Data Analysis

The correlations in Figure 1 show that $x4$ is strongly positively correlated with y and y_bin while $x5$ is strongly negatively correlated with y and y_bin . However, $x4$ and $x5$ are also strongly negatively correlated with each other. This raises concerns for multicollinearity. The plot also shows that $x3$ and $x4$ are correlated. Other than those mentioned there are a few weak correlations, but surprisingly most are around zero.

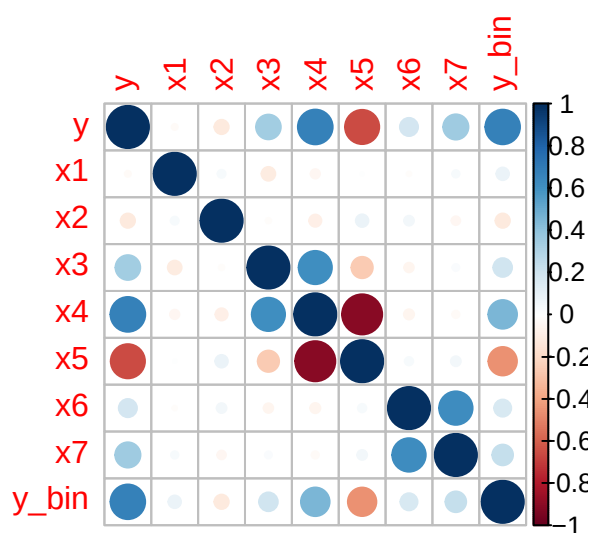


Figure 1: Correlations of Continuous Parameters

The scatterplots in Appendix Figure 4 show relatively the same information as Figure 1 with the addition of revealing the $\cos(x)$ relationship between x_6 and x_7 .

Our following linear regression and Principal Component Regression models assume y is normally distributed. We check that in Appendix Figure 5 by overlaying a normal distribution with the same mean and standard deviation as y over a histogram of y .

In Appendix Figure 6 we check the distribution of y grouped by x_8 . Each group doesn't appear as normally distributed as y is overall.

Similarly in Appendix Figure 7 we check the relationship between y_bin and x_8 . Here we notice that the classification groups are imbalanced which can make prediction more difficult. Another comparison between x_8 and y_bin can be found in Appendix Figure 8.

Regression Models

Linear Regression

We fit two basic linear regression models one with all predictors labeled “baseline linear” and another model with only x_4 , x_7 , and x_8 labeled “small linear”. The linear models assume that y is normally distributed which is a reasonable assumption from the EDA plots above. The linear models performed very well in the 10-fold cross validation compared to following regression models.

Random Forest Regression

Random forest is an extension and improvement upon bagging by using decorrelation. Bootstrap aggregation or bagging uses a tree based method and bootstrapping by taking repeated samples from the same training set and taking the average of the models, to make a new model that reduces the variance. The difference between bagging and random forest is that random forest uses, m , a random sample of our predictors from our total predictors p . A majority of the time m is found by evaluating the square root of p . Therefore, when there is a split in the tree, the model is not allowed

to consider all the predictors. This is important as it allows other predictors to essentially have a chance, as some of the splits will not consider our strongest predictors. At the end of the models' processes the average of our resulting trees will have less variability and therefore be a more reliable prediction. Random forest is especially helpful when we have many highly correlated predictors.

Boosted Trees Regression

Boosting builds off the premise of bagging with its own twist. In bagging the model is built on the single training set, on the other hand boosting is built sequentially. This means that if a tree is made, the next tree will be grown or made using the information of the previous tree. Any other tree made after this point will be grown from the information of all the trees before. Thus, boosting is not using bootstrap sampling. This process allows the model to avoid over fitting by learning slowly. Boosting does this by having three tuning parameters the number of trees, a shrinkage parameter, and the number of splits. In general, this model outperforms a random forest model.

Principal Component Regression (PCR)

Principal component regression works by first performing principal component analysis (PCA) to select a subset of principal components, then by using those principal components as the inputs for a linear regression model. PCA uses the singular value decomposition of the data to create a set of principal components, combinations of the original features, that maximize the variance of the data along that axis. To determine the number of principal components to keep, we use the “onesigma” approach given by the `selectNcomp()` function, which is based on the standard error of the CV results. Using 10-fold cross validation, we obtained a median RMSE of 9.8002 and a median MAE of 7.5488.

Neural Networks

Neural networks perform regression by learning the data via a set of nodes separated into input, hidden, and output layers. For our regression network, we use a single layer model with eight hidden nodes and a reLU activation function. Since Python's machine learning implementations are more developed than R, we chose to use PyTorch and Python 3.14.4 to run our neural networks. For training the model, we use the Adam optimizer as it is implemented in PyTorch with a learning rate of 0.01, 50 epochs of training, a batch size of 32 observations, and a MSE loss function. With 10-fold cross validation, we achieved a median RMSE of 10.4429 and a median MAE of 8.4182.

Plotting the 10-Fold Cross Validation Metrics

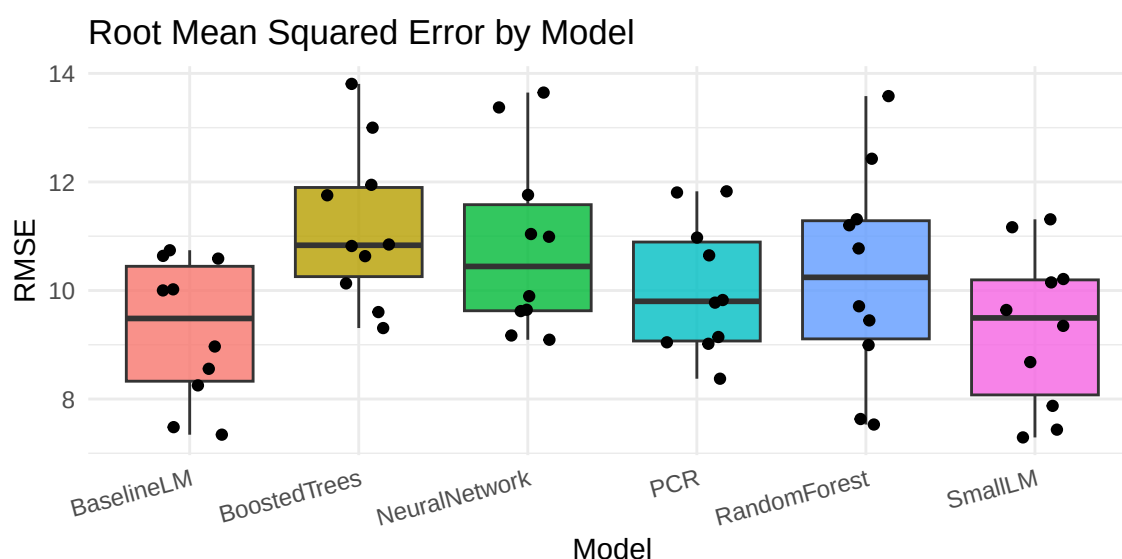


Figure 2: RMSE by Model for Each Fold

A smaller RMSE is better so the Baseline Linear and Small Linear models performed the best here. A representation of each fold can be found in [Appendix Figure 9](#).

The Mean Absolute Error was also considered which can be found in [Appendix Figure 10](#).

Classification Models

Standard Binary Logistic Regression

Using the `glm` function in R, we can fit a standard logistic regression using every predictor. Appendix Figure 11 provides the numerical summary of that model. Immediately the lack of significant predictors is apparent, with the majority of the independent variables holding p-values greater than 0.05. At this point, we need to check whether we have violated the assumptions of a standard logistic regression. Those assumptions are as follows: The outcome variable is binary, the data points in each group are independent (little to no clustering), the independent variables must be linearly related to the log-odds of the outcome, there must be little to no multicollinearity, no drastic outliers, and finally that we have a large sample size (Leung, 2025). A few of these we can check off quickly: First, our outcome variable y_{bin} is binary. Next, our sample size of 200 is generally considered large enough. Now, starting with checking for outliers and clustering, we can plot the individual predictor variables against the binned y values, whose plots are in Appendix Figure 12.

As seen in Appendix Figure 12, none of the values from the predictor variables have extravagant outliers, nor do they cluster dramatically in any scatterplot. At the bottom right of Appendix Figure 12 we have the plot of y_{bin} vs x_8 , which is different from the rest because x_8 is a categorical variable. There's no outliers to be seen here either, because values of x_8 can only be *A*, *B*, *C*, or *D*. Thus, we've met two more of our assumptions.

The next assumption we need to check is that our independent variables need to be linearly related to the log odds of the outcome. This can be done using the Box-Tidwell test. To do so, we must first define a new variable for each predictor variable (x_n) that is equal to $x_n * \log(x_n)$. Then, we define a logistic regression model, modeling y_{in} with x_n and $x_n * \log(x_n)$ as the only predictors. When we look at the model's summary, if the $x_n * \log(x_n)$'s p-value is significant (< 0.05), then we cannot assume x_n is linearly related to the log odds of the outcome (Leung, 2025). These steps are repeated for each predictor variable, as shown in the below figure. Note that $newx_n$ represents the $x_n * \log(x_n)$ value for the n th predictor variable.

In Appendix Figure 13, we notice that for predictors x_6 and x_7 , we reject our assumption that

they are linearly related to the log-odds of y_{bin} due to their low p-values. This means that for the actual model, we must transform them slightly instead of simply using them as they come from the dataset. To resolve the final assumption about standard logistic regression models, that there is little to no multicollinearity between predictors, we first need to create a model using all of our predictors. The only exception to this model is x_6 and x_7 , which will be transformed slightly due to our findings earlier. Upon doing so, we are left with the model in Appendix Figure 14.

The serious lack of significant predictors is worrying, but a potential solution to that comes from dealing with multicollinearity within the model, if it exists. We can check using the “Cars” package in R, followed by the `vif()` function, with the above model as the input to it. Doing so gives the result in Appendix Figure 15. Due to the categorical nature of x_8 , we don’t have exact VIF values, and instead must use the adjusted VIF scores, denoted in Appendix Figure 15 as column headed by $GVIF^{(1/(2*Df))}$. Adjusted VIF uses the same scale as the normal VIF but is compatible with categorical as well as numeric variables. As seen in Appendix Figure 15, there aren’t any adjusted VIF measurements above 10, but the lack of significant variables noted in Appendix Figure 14 implies that there still may be some issues. If we drop the highest adjusted VIF variable from the model, x_4 , and recreate the model, we get the following report from function `vif()` in Appendix Figure 16

This is a significant improvement from the previous model, implying minute amounts of multicollinearity. As a result of the nature of multicollinearity, any models with fewer predictors than the model represented in Appendix Figure 16 will also have the same or lower levels of multicollinearity. Therefore, our model has kept all the standard assumptions of standard logistic regression intact, and our final standard logistic regression model is as seen in Appendix Figure 17:

This model results in an 87.5% prediction accuracy, leaving room for improvement while still being quite decent at framing the data correctly. In our 10-fold cross validation models we test a baseline logistic model using all eight parameters and a log logistic model which uses the log of x_6 and x_7 and along with all other parameters except for x_4 .

Regularized Logistic Regression

This version of a logistic regression model adds in a penalty term that works to diminish overfitting. In order to find the error term, we needed to turn the dataset into a matrix and use the `cv.glmnet()` function to use cross-validation and get a model for potential values. For our error term we used the minimum lambda described by the cross-validation model and implemented it using Lasso (L1) regularization via the `glmnet()` function using our data in matrix form again. The resulting model returned an accuracy of 88%, which is barely better than that of the standard logistic regression model.

Logistic GAM

Known also as the logistic generalized additive model, is a method for categorizing your outcome variable when you suspect that its predictors don't follow a straight line (Daniel, 2025). We designate such variables by putting them inside the parenthesis of `s()`, and using the modeling function `gam()` from the “mgcv” package. Considering its similar setup to standard logistic regression, we chose to use the same predictor variables as we did in that model, but using the `s()` on every predictor except for `x8`, because `x8` is categorical. This led to a model seen in with an accuracy of 95.5%, a fantastic improvement upon the original logistic regression model.

K-Nearest Neighbors

K-Nearest Neighbors (KNN) assigns a class label to a new observation based on the most common class among its closest K neighbors. In our case, $K = 5$ was chosen using cross validation. Before fitting the parameters $x1$ through $x7$ were scaled to have a mean zero and a standard deviation of one to ensure all parameters contributed relatively equally. The categorical predictor $x8$ was encoded into four binary indicator columns in order to include it in the Euclidean distance measurement. Overall, with 10-fold cross validation KNN achieved a median accuracy of 84.17%, with a false negative rate at 0% for all but one fold.

Neural Networks

Since neural networks can also be used for classification, we applied them here as well. As with the regression neural network, we used a single hidden layer with eight nodes, the Adam optimizer, a batch size of 32, and 50 epochs, all implemented in Python. However, since using the MSE loss function would be nonsensical for classification, we instead used the cross entropy loss function. To this end, with 10-fold cross validation, we achieved a median accuracy of 0.8095, a median false positive rate of 0.1292, and a median false negative rate of 0.375.

Plotting the 10-Fold Cross Validation Metrics

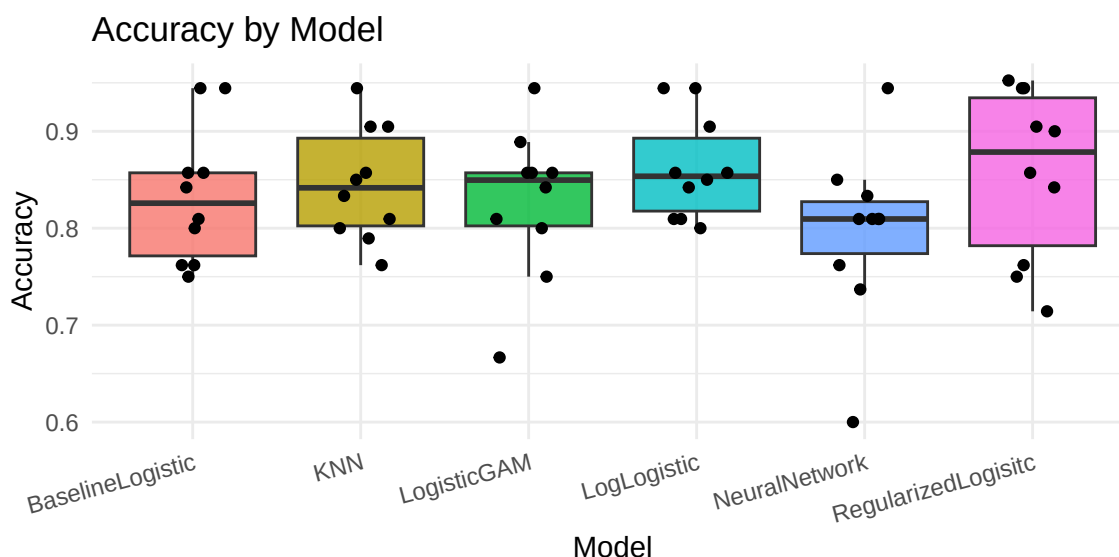


Figure 3: Accuracy by Model for Each Fold

A higher accuracy is better so the KNN, Log Logistic, and Regularized Logistic perform the best here. The False Positive Rate and False Negative Rate were also considered in Appendix Figure 19 and Appendix Figure 20 respectively. Additionally the plot of accuracy show for each fold can be found in Appendix Figure 21.

Conclusion

We were given seven numerical predictors labeled x_1 through x_7 and one categorical predictor x_8 for predicting a response variable y . Additionally y was transformed into y_{bin} by setting $y_{bin} = 1$ if $y \geq 70$ and 0 otherwise. Initial exploratory data analysis revealed that predictors x_3 , x_4 , x_5 , and x_7 were well correlated with y and y_{bin} , with x_4 and x_5 correlated with each other. We applied six regression models to predict y and six classification models to predict y_{bin} . Then 10-fold cross validation was used to assess the models. Root Mean Squared Error and Mean Absolute Error was assessed for the regression models and Accuracy, False Positive Rate, and False Negative Rate were used to assess the classification models. For regression the linear models performed best while for classification k-nearest neighbors, regularized logistic, and log logistic models all performed well.

Appendix

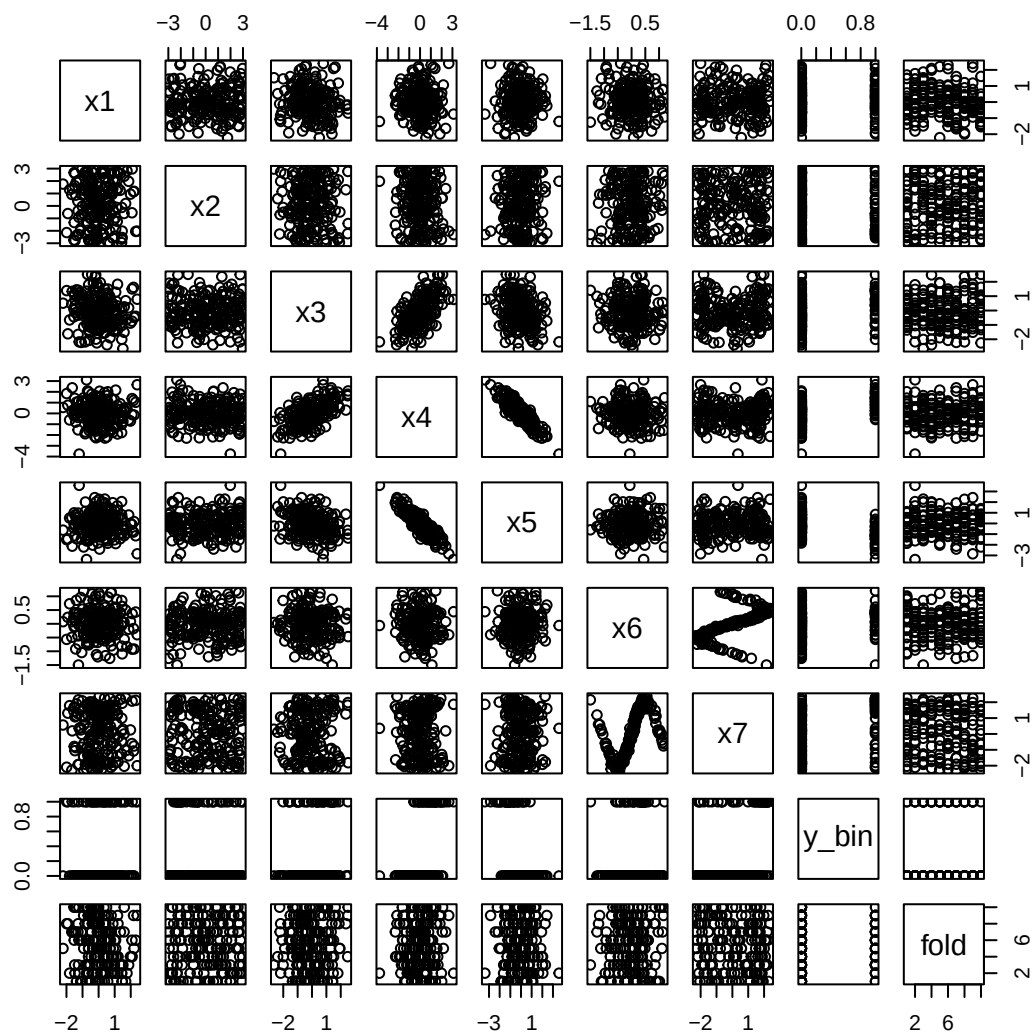


Figure 4: Scatterplots of Continuous Parameters

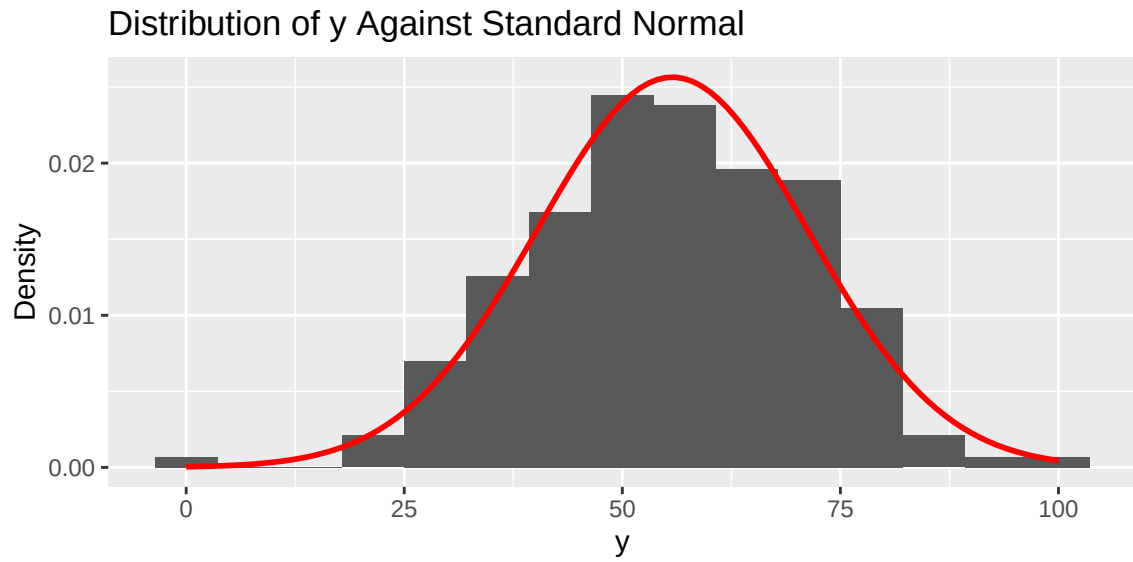


Figure 5: Histogram of y

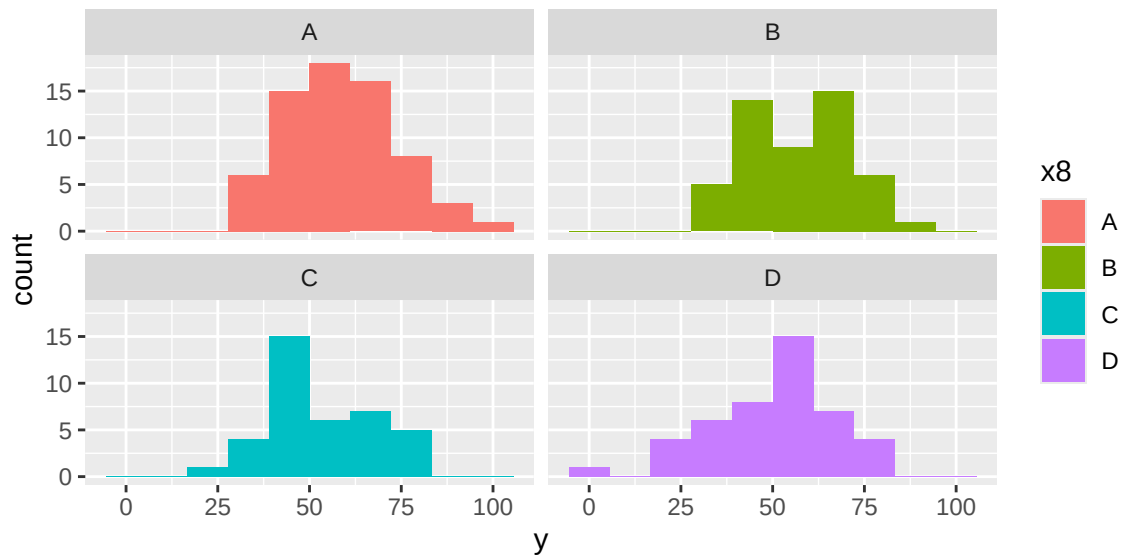


Figure 6: Histograms of y by x8

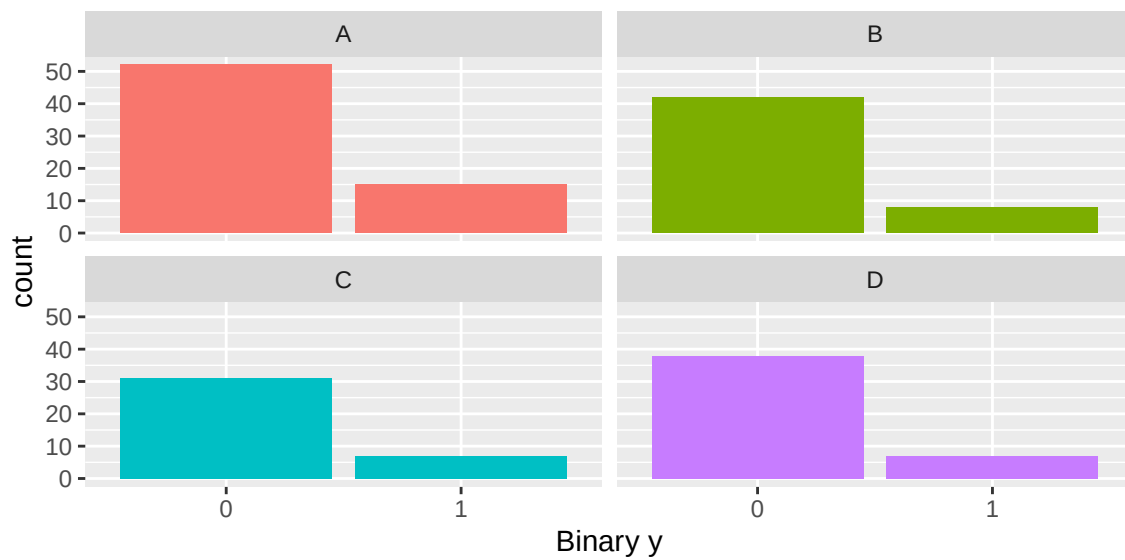


Figure 7: Barplots of binary y by x8

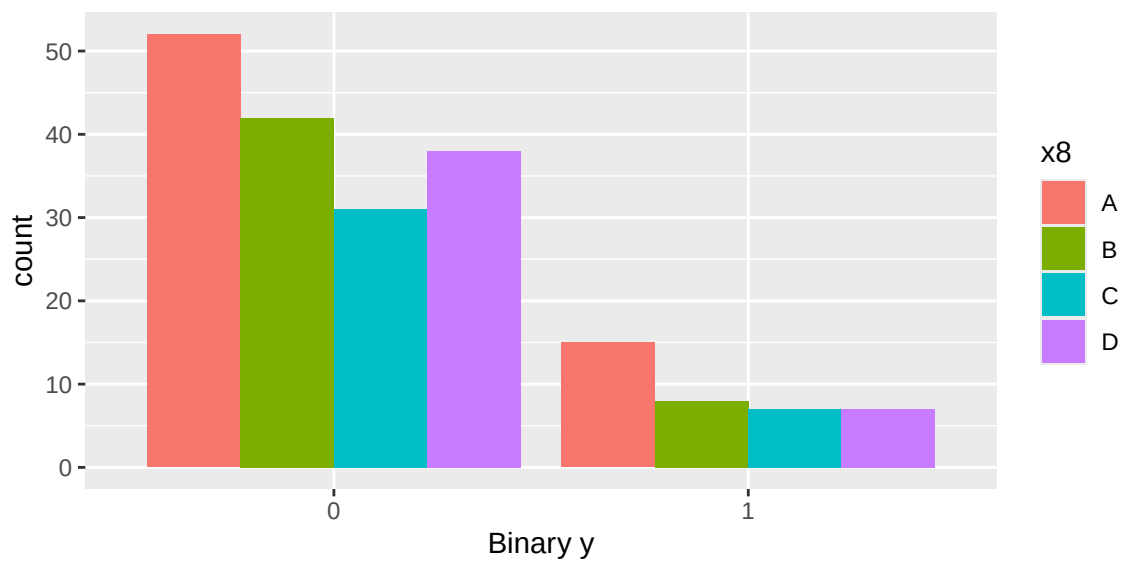


Figure 8: Grouped barplot of binary y by x8

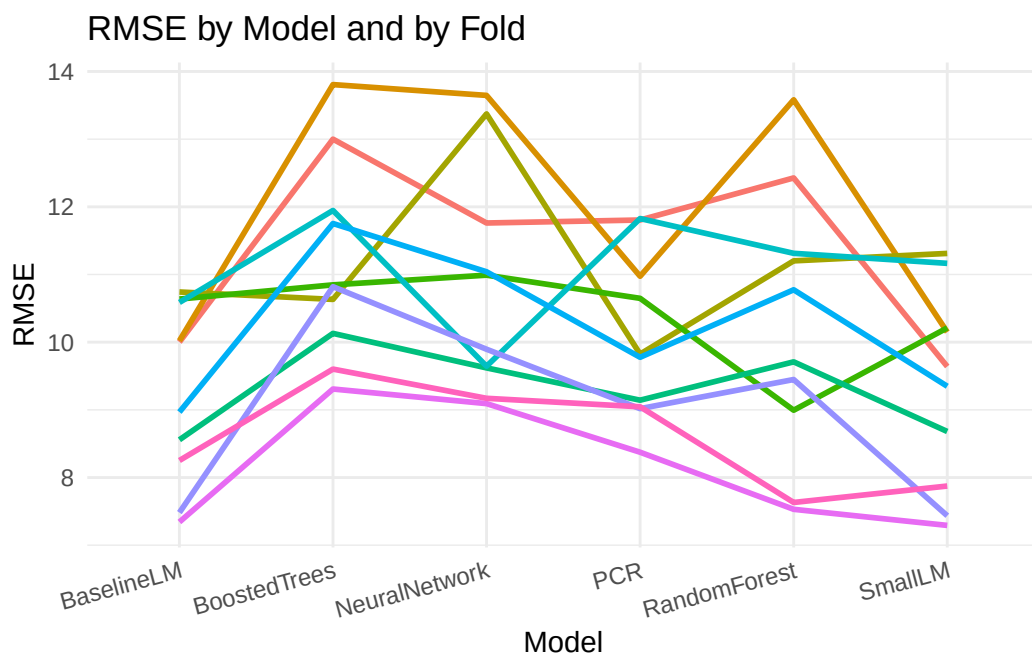


Figure 9

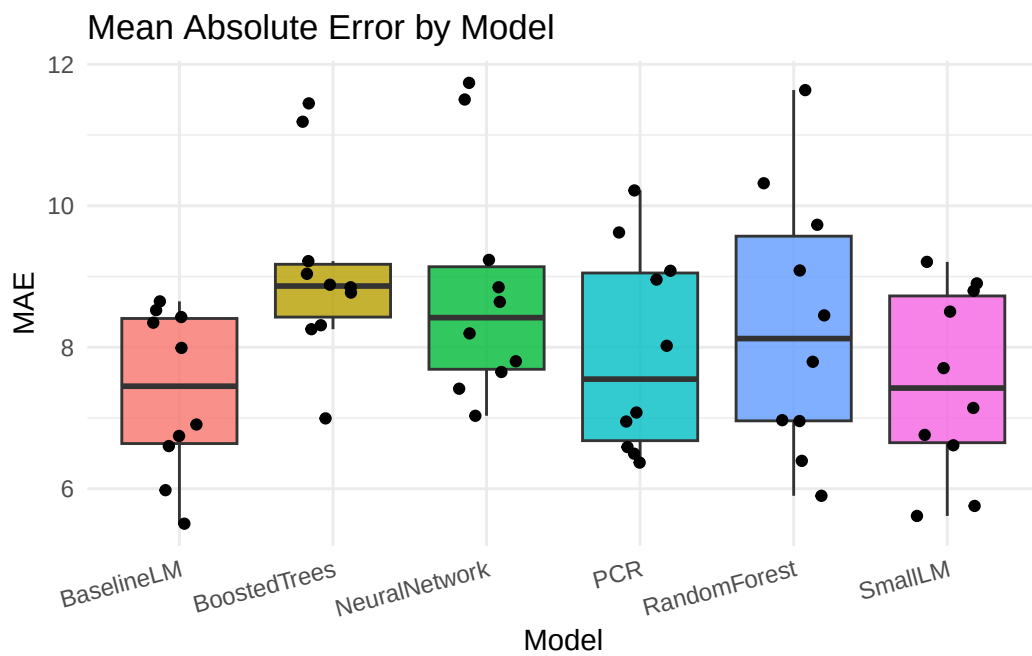


Figure 10

```

Call:
glm(formula = ybin ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8, family = binomial,
    data = FP_di)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.3626     0.5001  -4.724 2.31e-06 ***
x1             0.5494     0.2742   2.004 0.045122 *
x2            -0.1866     0.1575  -1.185 0.235959
x3            -0.1923     0.7719  -0.249 0.803300
x4             1.7534     1.6982   1.032 0.301851
x5            -0.7956     1.4050  -0.566 0.571203
x6             0.2261     0.5576   0.406 0.685090
x7             0.9472     0.2656   3.566 0.000363 ***
x8B          -1.7534     0.7377  -2.377 0.017461 *
x8C          -0.5180     0.7104  -0.729 0.465833
x8D          -1.8050     0.8173  -2.208 0.027211 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Figure 11: Summary of Initial Logistic Model

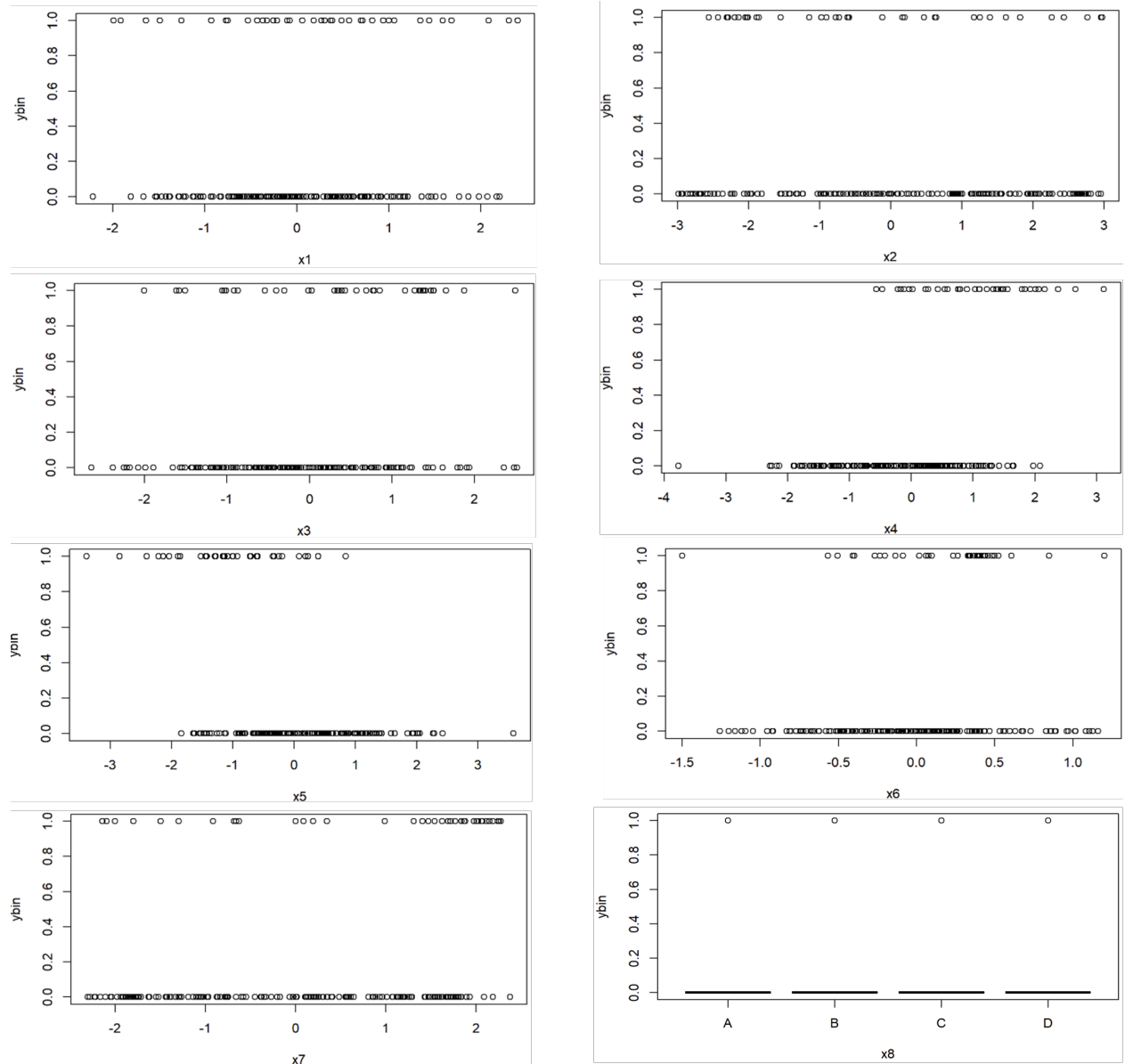


Figure 12: Binned y values vs individual predictors


```
Call:
glm(formula = ybin ~ x1 + newx1, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.34223    0.86140  -1.558   0.119
x1           0.04347    1.05418   0.041   0.967
newx1        0.54778    1.02683   0.533   0.594
```

```
Call:
glm(formula = ybin ~ x2 + newx2, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.0057    1.0001  -1.006   0.315
x2          -0.9035    1.1473  -0.788   0.431
newx2        0.6501    0.8763   0.742   0.458
```

```
Call:
glm(formula = ybin ~ x3 + newx3, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.737    1.032  -2.654  0.00796 **
x3           2.083    1.206   1.727  0.08412 .
newx3       -1.400    1.115  -1.256  0.20911
```

```
Call:
glm(formula = ybin ~ x4 + newx4, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.9964    0.9628  -2.074  0.0381 *
x4           1.1803    1.1458   1.030  0.3030
newx4        0.6285    1.1704   0.537  0.5913
```

```
Call:
glm(formula = ybin ~ x5 + newx5, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.489    1.477  -1.008   0.313
x5          -3.005    1.869  -1.608   0.108
newx5       -1.347    4.089  -0.329   0.742
```

```
Call:
glm(formula = ybin ~ x6 + newx6, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.9941    0.9691  -3.090  0.0020 **
x6           1.2975    1.0345   1.254  0.2098
newx6       -5.1561    2.3349  -2.208  0.0272 *
```

```
Call:
glm(formula = ybin ~ x7 + newx7, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.5459    0.8301  -0.658  0.51077
x7          -2.0577    1.0659  -1.931  0.05354 .
newx7        3.4799    1.1221   3.101  0.00193 **
```

Figure 13: Summaries of Box-Tidwell Models

```
Call:
glm(formula = ybin ~ x1 + x2 + x3 + x4 + x5 + log(x6 + 3) + log(x7 + 3) + x8B + x8C + x8D, family = binomial, data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -5.5560    1.6262  -3.417 0.000634 ***
x1           0.5522    0.2683   2.058 0.039555 *
x2          -0.2131    0.1545  -1.379 0.167953
x3          -0.1409    0.7558  -0.186 0.852110
x4           1.7530    1.6602   1.056 0.290992
x5          -0.7173    1.3766  -0.521 0.602309
log(x6 + 3)  0.9907    1.5041   0.659 0.510111
log(x7 + 3)  2.2197    0.6878   3.227 0.001250 **
x8B         -1.5674    0.7053  -2.222 0.026252 *
x8C         -0.4724    0.7020  -0.673 0.501025
x8D         -1.5639    0.7714  -2.027 0.042637 *
```

Figure 14: Improved Standard Logistic Regression Model Summarized

	GVIF	Df	GVIF ^{1/(2*Df)}
x1	1.136026	1	1.065845
x2	1.109377	1	1.053270
x3	11.479255	1	3.388105
x4	27.360025	1	5.230681
x5	17.578727	1	4.192699
log(x6 + 3)	1.378928	1	1.174278
log(x7 + 3)	1.911865	1	1.382702
x8	1.533878	3	1.073903

Figure 15: Initial VIF Report

	GVIF	Df	GVIF ^{1/(2*Df)}
x1	1.117639	1	1.057184
x2	1.094885	1	1.046367
x3	1.257233	1	1.121264
x5	1.470021	1	1.212444
log(x6 + 3)	1.379700	1	1.174606
log(x7 + 3)	1.928150	1	1.388579
x8	1.504567	3	1.070455

Figure 16: Improved VIF Report

```
Call:
glm(formula = ybin ~ x1 + x2 + x3 + x5 + log(x6 + 3) + log(x7 +
3) + x8, family = binomial, data = FP_di)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-5.5091	1.6268	-3.386	0.000708	***
x1	0.5339	0.2623	2.036	0.041757	*
x2	-0.2057	0.1530	-1.344	0.178817	
x3	0.6214	0.2489	2.496	0.012551	*
x5	-2.1350	0.3967	-5.382	7.37e-08	***
log(x6 + 3)	0.8931	1.5025	0.594	0.552246	
log(x7 + 3)	2.2867	0.6810	3.358	0.000786	***
x8B	-1.6467	0.7050	-2.336	0.019500	*
x8C	-0.4669	0.6988	-0.668	0.504031	
x8D	-1.5198	0.7598	-2.000	0.045481	*

Figure 17: Summary of Final Standard Logistic Regression Model

ybin ~ s(x1) + s(x2) + s(x3) + s(x5) + s(log(x6 + 3)) + s(log(x7 + 3)) + x8					Approximate significance of smooth terms:			
Parametric coefficients:					edf	Ref.df	Chi.sq	p-value
(Intercept)	Estimate	Std. Error	z value	Pr(> z)	s(x1)	2.669	3.316	6.999 0.10236
x8B	-1.398	1.151	-1.214	0.22479	s(x2)	1.000	1.000	3.589 0.05817 .
x8C	-0.514	1.173	-0.438	0.66126	s(x3)	8.834	8.978	15.206 0.08415 .
x8D	-3.373	1.302	-2.592	0.00955 **	s(x5)	4.869	5.709	19.226 0.00344 **
---					s(log(x6 + 3))	1.000	1.000	0.001 0.97595
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1					s(log(x7 + 3))	6.065	7.161	19.449 0.00777 **

Figure 18: Logistic GAM Model Summary

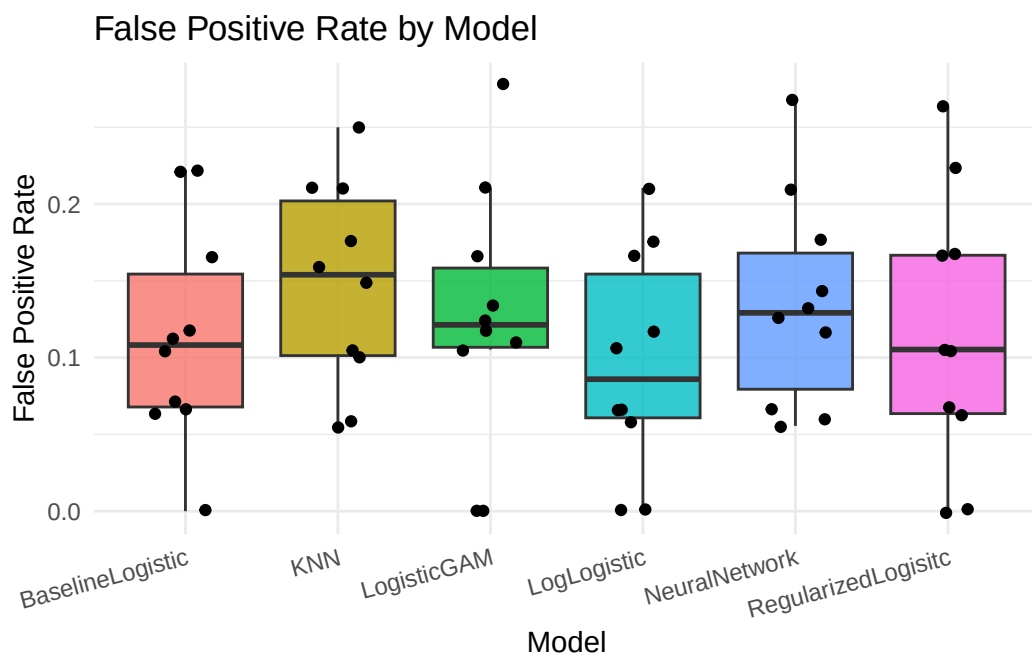


Figure 19: False Positive Rate by Model for Each Fold

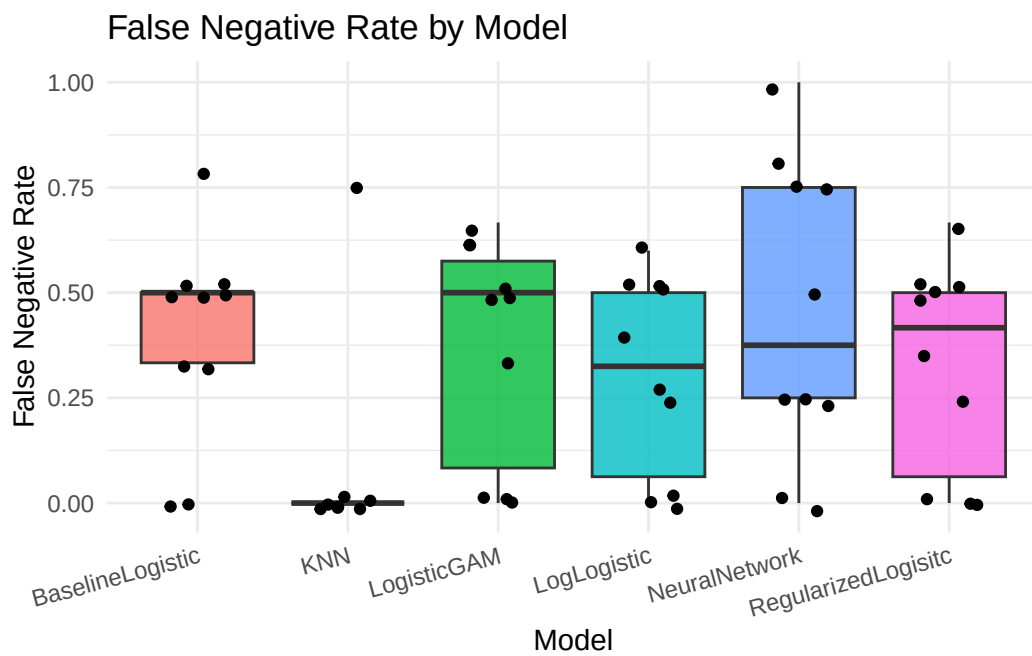


Figure 20: False Negative Rate by Model for Each Fold

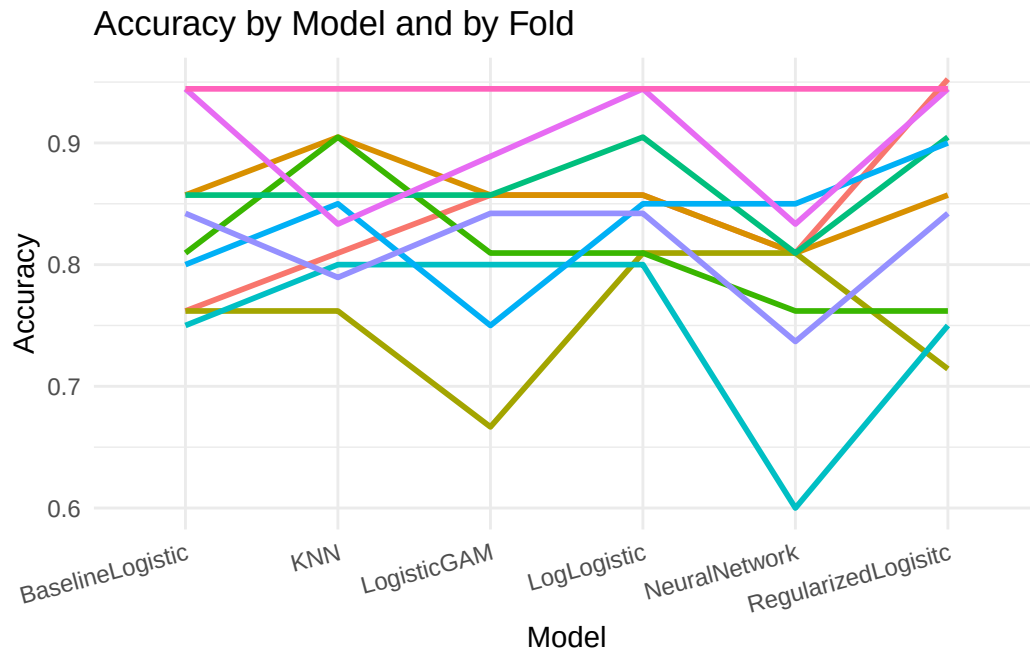


Figure 21: Accuracy by Model and by Fold

References

- Daniel. (2025). Understanding Generalized Additive Models. RPubS. <https://rpubs.com/AfroLogicInsect/GAM>
- Leung, K. (2025, March 5). Assumptions of logistic regression, clearly explained. Towards Data Science. <https://towardsdatascience.com/assumptions-of-logistic-regression-clearly-explained-44d85a22b290/>
- Leung, K. (2025, March 5). Assumptions of logistic regression, clearly explained. Towards Data Science. <https://towardsdatascience.com/assumptions-of-logistic-regression-clearly-explained-44d85a22b290/>
- Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie. (2009). The Elements of Statistical Learning. <https://hastie.su.domains/ElemStatLearn/>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32.